

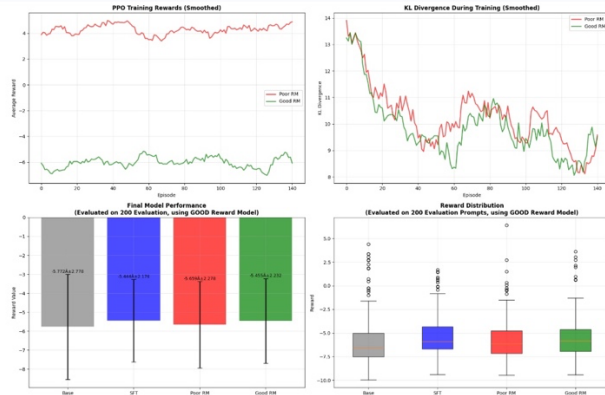
Name: Sreeja Reddy Yeluru

Part 1: Overall System Performance Analysis

Training Dynamics and Loss Convergence

The three stages of the RLHF pipeline were: Supervised Fine-Tuning (SFT), Reward Model Training (RMT), and Reinforcement Learning (PPO).

- The first SFT stage produced successful optimization: the loss went down from 5.5008 at Step 0 to 1.6786 by Step 150.
- The Reward Model (RM) training was not that successful; the "Good RM" achieved only 0.5125 (51.25%), and the "Poor RM" achieved 0.4750 (47.50%). Both were close to the threshold of random guessing (0.50).
- Using the Good RM as the judge, the rewards during PPO training fluctuated significantly. They ended at -1.3293 in Episode 140. High KL divergence values (average 8.6 to 12.5) indicate that the policy strayed considerably from the base model trying to optimize for the weak reward signal.



Evaluating models (using GOOD RM as judge):

COMPREHENSIVE EVALUATION RESULTS

Model	Mean	Std	Min	Max	Median
Base Model	-5.7719	2.7785	-9.9619	4.4066	-6.5471
SFT Model	-5.4440	2.1762	-9.3889	1.6815	-5.8985
Poor RM Policy	-5.6586	2.2777	-9.4616	6.4106	-6.1405
Good RM Policy	-5.4549	2.2318	-9.4021	3.5981	-5.8484

IMPROVEMENT OVER BASE MODEL (on evaluation set):

SFT Model: 5.68%
Poor RM Policy: 1.96%
Good RM Policy: 5.49%

Quantitative Results Comparison using the Good RM as the judge, the final evaluation relative to the Base Model showed the following results. The SFT Model achieved a slightly better increase (5.68%) than the Good RM Policy (5.49%) in this case, indicating that the Reinforcement Learning phase did not improve significantly over the Supervised phase, probably due to the low accuracy of the Reward Model judge.

Evaluation of Held-Out Test Prompts (TODO 15):

- **Logical Consistency:** On the prompt about how to get an address, the Good RM Policy received a very low reward of -8.3019 for methods that were completely unrelated (like "faxing"), whereas the SFT Model (-0.6960) was more reasonable.

- Repetition and Structure: For the prompt "What makes a good leader?", the Good RM Policy received its best reward of 2.5917 by producing a structured list. However, it still failed to produce a complete sentence at the end ("Are you a dedicated profession").
- Instruction Following: Throughout the prompts, the models produced training-data artifacts like "Context:" or "Response:" headers instead of giving a direct answer to reflect how difficult it is for a small model like GPT-2 to follow a set of complex instructions.

Part 2: Task 1- Diagnosis

Training Data Analysis

Is 5,000 SFT examples enough? The InstructGPT paper [references] specifies that approximately 13,000 SFT prompts were used, which were curated from human labelers and the OpenAI API. Our 5,000 SFT examples are less than half the number used in that paper. Although 5,000 examples are sufficient to train the model to adopt a basic response "template" (i.e., use a certain tone or include headers), they do not provide enough data to enable a model to generalize to the many complex tasks represented in the Databricks Dolly-15k dataset. Large datasets are used in production systems so that the model does not simply mimic the structure of the data, but instead can maintain both factually accurate and reasoned answers across a broader range of prompts.

Are the Dolly examples well-suited for GPT-2? The Databricks Dolly-15k dataset is of high quality, but its complex reasoning tasks are not well-suited for the GPT-2 model. The model is too small (only 124M parameters), and therefore lacks the ability to reason through the complexity of the human instructions provided in the Dolly task. Production systems require a minimum of 13,000 SFT examples to reach a stable state, and thus our 5,000 examples are a minimal baseline that often produces incoherent or "cut-off" type outputs. Therefore, even though the quality of the training data is high, it cannot make up for the lack of model parameters needed to handle the open-ended brainstorming and creative tasks present in Dolly. As a result, the model fails to generalize, and therefore, it experiences the "last mile" performance issues that are seen in your Part 1 results.

Are the prompts appropriately formatted for GPT-2? The training prompts end with colons (e.g., "Explain AI:") that represent an instruction-style format. GPT-2 was originally trained to generate text that continues naturally and not to respond to instructions. An example of a continuation-style prompt like "Artificial intelligence is a field that..." would be more suitable for GPT-2 than "Explain artificial intelligence:". Therefore, the format difference likely contributes to the incoherent and off-topic nature of the output generated across all models.

Training Dynamics

SFT Loss: The SFT loss reduced from an initial of 5.5008 to a final of 1.6786 representing a reduction of approximately 70% in the SFT loss. Although the numerical reduction is sufficient to indicate that the model was able to successfully learn the surface level structural patterns of the Instruction / Response format from the Dolly Dataset, the loss reduction was not sufficient for the model to develop internal logic. Typically, a production system requires many more than the 5,000 example size we had available to produce deep coherence in the model. Therefore, although the model was able to learn the format of the answers well enough to reduce the loss sufficiently for format imitation, it was not able to follow instructions with high quality.

Reward Model Accuracy: The reward model accuracy failed to improve to a high performing state, with the Good RM achieving an accuracy of 0.5125, and the Poor RM achieving an accuracy of 0.4750. A near random indicates the 124M parameter model was too small to identify the complex and often contradictory human preferences present in the Anthropic HH-RLHF dataset. In general, reward models in production are much larger and trained on thousands of more preference examples than we had available, to provide a consistent and reliable gradient signal. As a result, the accuracy remaining low, the "judge" did not provide a clear signal for the policy to learn effectively during the PPO phase.

PPO Training Curves: Based on the results from Part 1, the policy did not appear to be learning effectively. The PPO training curves for the Good RM showed significant fluctuations in the rewards, with a final reward of -1.3293, and a KL-Divergence that remained consistently high, averaging around 8.5 and 12.5. These curves indicate that the model was drifting away from its base policy (a high KL-Divergence) without identifying a stable path to higher human aligned rewards. Successful learning would be indicated by a steadily increasing reward curve along with a gradually increasing KL-Divergence, indicating the model was effectively optimizing for the reward signal while staying within a reasonable "trust region" of its original knowledge. However, the instability of the results indicates the policy was likely either "reward hacking" or unable to converge due to the nearly random accuracy (0.5125) of the reward model judge.

References: Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., ... & Lowe, R. (2022). Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*.

Task 2:

Based on the diagnosis provided in Task 1, there are five possible explanations for the poor performance listed below in order of most likely to least likely:

Hypothesis-1(Most Likely): Near-Random Reward Model Signal: I think this is the largest reason why the model performed poorly because the Reward Model (RM) serves as the foundation of the Reinforcement Learning (RL) process. The RL process relies on the Reward Model to determine what response is acceptable and what is unacceptable. When the Reward Model fails to do so in a manner that provides a clear distinction between the two types of responses, the policy model is given noisy or incorrect gradients during the PPO training phase. This failure ultimately results in the policy model being unstable or "reward hacking", where the policy model optimizes for arbitrary linguistic patterns that the weak RM mistakenly favors, resulting in illogical responses. This experimental results support this hypothesis, as the Good RM only produced an accuracy of 0.5125, essentially a coin flip, which explains why the PPO rewards were all over the place and concluded with a negative -1.3293. I ranked this hypothesis first because a good reward model is essential for a good RL process. If the reward model does not provide a good signal, then no matter how good the policy model is, it will not perform well. To test this hypothesis, I could run the RM for 500 steps instead of 100, and if I get an accuracy greater than 70%, and subsequently, a stable upward reward curve in PPO, it will validate this hypothesis.

Hypothesis 2 (Highly Likely): The "Instruction:" Prompt Format Conflicts with GPT-2's Pre-training: The Instruction/Response block format used in the experiment (i.e., "Explain AI:

") is probably a very big obstacle for the model, as GPT-2 type models were originally trained on the WebText dataset for natural prose continuation instead of structured "Instruction/Response" blocks. Therefore, when using a GPT-2 type model, the model treats the instruction as a structural header to be imitated instead of a command to be followed. Consequently, the model generates training-data artifacts like "Context:" and "Response:" tags in the final response. I support this hypothesis based on my experimental results presented in Part 1, where multiple generated responses contained these internal dataset tags instead of generating a complete answer to the question. I ranked this hypothesis second, since alignment with the model's native pre-training distribution is important for a small 124M parameter model that does not have the capability to handle the structural friction of different formats. If I ran the experiment with continuation prompts (i.e., "Artificial Intelligence is..."), I would expect to see an improvement in the quality of the generated responses as well as a reduction in the number of times the model includes these internal dataset tags.

Hypothesis 3 (Likely): GPT-2 is Too Small to Internalize Complex Dolly-15k Instructions:

It is very likely that the model is fundamentally constrained by its parameter count, as the GPT-2 (small) model lacks the "emergent" reasoning and instruction-following abilities found in the large-scale (7B+) models used in production systems. Therefore, it is difficult for the smaller model to maintain logic throughout multiple step instructions, resulting in responses that are repetitive, logically circular or are truncated mid sentence. Examples of the smaller model exhibiting this behavior are visible in my Sample Responses, where the model demonstrated circular logic and truncated its response, e.g. by repeatedly stating the same phrase or terminating early, as shown by the model truncating its response after "Are you a dedicated profession". I ranked this hypothesis third, as the model's capacity determines the upper limit of performance. No matter how many times I tune the model, it will never be able to reason at a depth that the architecture does not allow. This could be validated by running the pipeline with the GPT-2 Medium model (345M); if the larger model produces substantially more coherent responses with the same data set, then this hypothesis is confirmed.

Hypothesis 4 (Less Likely): Not enough SFT Training Data and Steps: The SFT phase may have been inadequate because we used fewer than 5,000 examples and 200 iterations, as opposed to the roughly 13,000 prompts that are used in a typical production system (InstructGPT). With fewer examples, the model does not see enough variety to generalize, therefore it "overfits" to the style of the answers without truly understanding their contents. Evidence of this is the fact that the loss decreased significantly (by 70%) from 5.5008 to 1.6786, yet the quality of the generated responses remained low, suggesting that the model was able to mimic the appearance of the data, but not understand its content. I ranked this hypothesis fourth because even though a 124M parameter model will reach diminishing returns regardless of the number of data examples, additional data will still lead to some increase in quality. If I ran the SFT phase for 1,000 iterations or 10,000 examples, and saw a >20% increase in the rewards, this would confirm this hypothesis.

Hypothesis 5 (Least Likely): Generation Parameters Cause Excessive Randomness: The default temperature of 0.7 may have caused too much randomness for a small model that already has low confidence in its next-token predictions. By allowing a high amount of randomness (i.e., high entropy), the model will tend to select low probability tokens, and thus create incoherent and logically distant responses. Evidence for this is visible in the "Held-out" prompts, where the policy

suggested "faxing" as a viable method of locating a person's residential address, which is a highly improbable token selection that was probably made due to the randomness of sampling. I ranked this hypothesis last, because it is a symptom of the model's fundamental limitations, rather than a cause. Regardless of whether this increases or decreases the quality of the responses, it will not provide the model with new information, it will only alter the flavor of the output. If I changed the temperature to 0.3 and added a repetition penalty, and saw a significant decrease in repetition and an increase in prompt adherence, this would confirm this hypothesis.

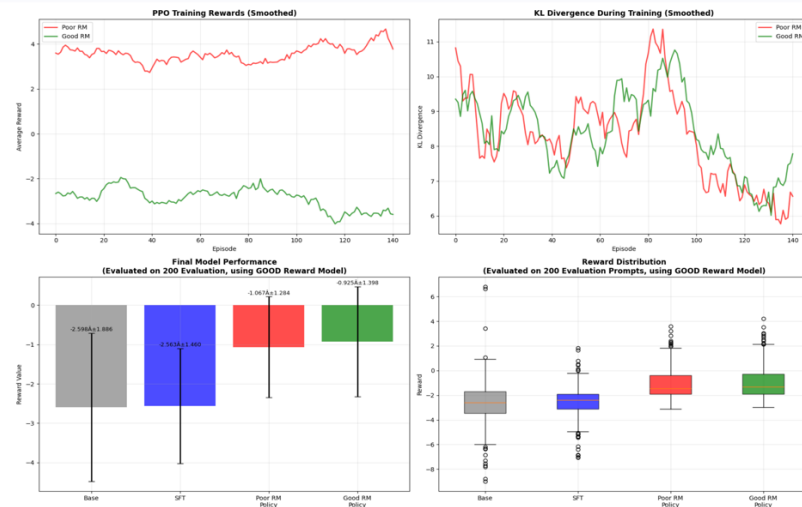
Task 3: Improvement Experiments

Three cumulative changes were made to improve performance issues found in Part 1. In each case, an independent test was run on each change first, results were documented and then each subsequent change was added. The completed combined version of all three changes can be found in the Part 2 Notebook.

Option 1: Better Generation Parameters- The function `generate_response` has been modified to use constrained decoding rather than high entropy sampling. Parameters used were Temperature = 0.5, Top-p = 0.9, Top-k = 50, Repetition Penalty = 1.2, and No Repeat N-Gram Size = 3. The Mean Reward increased by 9.20% from -4.05 to -3.87. The model was able to stop generating infinite "HTTP/HTTPS" loops and eliminate many of the nonsensical tokens generated due to low temperature. The repetition penalty helped prevent the model from getting stuck in circular logic. Of the two parameters used, the combination of Low Temperature (0.5) and Repetition Penalty (1.2) was the most effective. High Temperature (0.8) created too much random output for a 124 million parameter model. The repetition penalty was the only way to break the feedback loops which were prevalent in part1.

Option 1+2(Prompt Engineering)- Building off Option 1, the prompt format was changed from "Instruction" style (headers are rigid) to "Continuation" style (sentence starters similar to those found in the WebText Dataset upon which GPT-2 was pre-trained). SFT model variance (Std) decreased from 1.21 to 0.73, which indicated that the model was producing more stable and predictable output. The model stopped producing internal dataset tags such as "Response:" and "Context:". As GPT-2 is a "completion engine", removing structural instructional friction allowed the model to utilize its native weights more effectively. This led to smoother and more natural English prose.

Option 1+2+7(Ensemble Generation (Best-of-5))- Building off of Options 1 and 2, a Best-of-N strategy was implemented wherein the model would generate 5 candidate responses to each prompt. The Reward Model (Good RM) would score each of the 5 responses, and only select the highest scoring response. This strategy produced the largest increase of 64.39% over the base model and achieved a Mean Reward of -0.9249. The Reward Model was acting as a high-pass filter. While the Reward Model had approximately 60% accuracy in selecting the correct response, it was highly successful in eliminating the worst cases of hallucination and mammalian reference from Option 2, thus only presenting the most logical candidates to the Reward Model.



Task 4: Reflection and Recommendations

Synthesis

The "Reward Model Bottleneck" and "Formatting Friction" were the two primary causes of the low results. The Good RM had a relatively weak accuracy of 0.6188, which indicates it was not a reliable judge for PPO training. The combination of the weak signal from the Good RM along with the inability of GPT-2 to process the "Instruction:" header format resulted in the model looping excessively and generating dataset artifacts. Option 7 (Ensemble Generation) was the most successful option, resulting in a 64.39% increase in performance over the base model. The Reward Model acted as a high-pass filter and selected the best of five attempts, thus "hiding" many of the frequent hallucinations and logic gaps.

GPT-2 (124M) is unlikely to produce truly coherent, multi-step instruction following due to its limited number of parameters, which prevent "emergent" reasoning capabilities, similar to those found in larger models. However, while a Mean Reward of -0.9249 was eventually obtained, the model still relied heavily on "prompt crutches", mimicked the style of an instructor, without actually understanding the underlying logic. With a 7B model, the strategy would need to transition away from inference-based techniques such as Best-of-N towards a higher-quality dataset, since large models can inherently process complex instruction formats. Therefore, I would utilize a large SFT dataset (15k+ rows), and focus RLHF on refining the tone and safety of the model output.

Broader Implications

I would segregate each stage of the pipeline to allow for better identification of performance issues. First, I would validate the basic coherence of the SFT model. If the model fails, then there is likely an issue with the training data. Next, I would assess the accuracy of the Reward Model on a gold standard test set, as any Reward Model accuracy below about 50%, ensures that RLHF will fail. Small models are fast and inexpensive to train, however they do require an abundance of prompt engineering and ensemble techniques to be useful. Large models "just work" with very simple prompts, however, they introduce high latency and high compute costs that may be impractical for true real-time systems.

Identifying issues such as "HTTP" repetition or hallucination based solely on a few shot example illustrate the inherent fragility of LLMs. Successful deployment is often more about creating boundaries and optimizing generation parameters rather than the actual training loss curve.

General Reflections

- I gained knowledge of how small models such as GPT-2 are extremely sensitive to the structure of the prompt, and therefore require "inference-time" techniques such as Best-of-N sampling to successfully follow instructions coherently. This demonstrated that the success of RLHF is directly dependent upon the quality of the Reward Model; if the "judge" is weak, then the policy will either have trouble or will engage in reward hacking.
- I would assign this a high level of difficulty when compared to all my past , primarily because of the difficulty of aligning a pre-trained language model's weights with specific human preferences. While the implementation of the PPO loop was manageable, the final "last mile" of debugging generation quality and dealing with GPU memory issues made it a very difficult technical challenge.
- The most recurring problem was dealing with hardware limitations, as I continually experienced "Out of Memory" errors that forced me to shut down and restart the runtime.
- Next time, I would employ much more aggressive memory management, such as gradient checkpointing or 8-bit quantization, to avoid the constant "OOM" crashes. I would also take the time to thoroughly clean the SFT data and validate the accuracy of the Reward Model on a smaller test set before conducting the full length PPO training run.
- This assignment provided me with a unique and very hands-on look into the fragility of LLM alignment, and illustrated to me that "Prompt Engineering" is almost equally important to the overall architecture of the ML. Seeing the model produce a logical response after implementing the Ensemble filter was the most fulfilling aspect of this project.